SMARTCAMPUS — PLATEFORME MICROSERVICES

Gestion des Résidences

Documentation technique — Backend NestJS (Chambre · Demande · Contrat) & Frontend React

NestJS 10 + TypeORM + PostgreSQL | Vite + React 19 + Tailwind | Swagger / OpenAPI | Event-driven interne

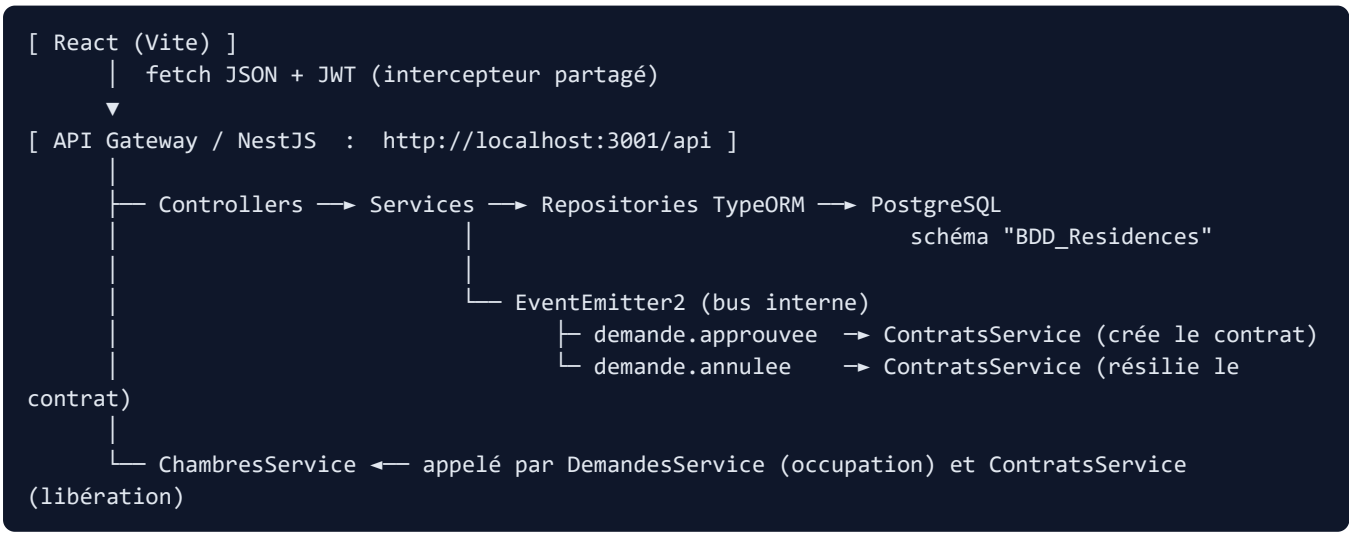
1. Contexte et périmètre

Ce document décrit le travail réalisé pour le module **« Gestion des Résidences universitaires »** de la plateforme SmartCampus. Le périmètre couvre :

- **Backend** : une application **NestJS** organisée en trois modules fonctionnels jouant le rôle de microservices logiques — **Chambre** (socle), **Demande** (cycle de vie des demandes), **Contrat** (créé à l'approbation).
- **Frontend** : les pages du module Résidences intégrées à l'application React partagée de l'équipe (routes `/residences/*`), branchées sur les API ci-dessus.
- **Base de données** : PostgreSQL, schéma dédié `BDD_Residences`.

> **Point important :** le frontend de l'équipe est une application **Vite + React 19 + React Router** (et non Next.js). Le module Résidences y existait sous forme de maquette 100 % mockée avec un modèle différent (résidences/lits/tickets). Il a été **réécrit pour refléter le contrat réel du backend** (Chambre / Demande / Contrat), tout en conservant le mode démo mocké et le design system existant (navy + violet).

2. Architecture d'ensemble



Les trois services partagent la même base (décision validée : « shared DB + injection de services »). Les dépendances s'expriment par **injection de services NestJS** et par **événements de domaine**, ce qui permettra une extraction en microservices HTTP indépendants sans changer les règles métier.

3. Backend — Phase 1 : Chambre Service

3.1 Entité `Chambre`

Champ	Type	Contraintes
`id`	uuid (PK)	généré automatiquement
`numero`	varchar(50)	unique par bâtiment
`type`	enum	`SIMPLE | DOUBLE | STUDIO`
`capacite`	int	1–4 ; cohérente avec le type (SIMPLE = 1, DOUBLE = 2, STUDIO ≤ 2)
`disponible`	boolean	défaut `true`
`batiment`	varchar(100)	contrainte d'unicité `(batiment, numero)`
`etage`	int	≥ 0
`createdAt / updatedAt`	timestamptz	audit automatique

3.2 Endpoints

Méthode	Route	Description	Erreurs
POST	/chambres	Créer une chambre	400 capacité incohérente · 409 doublon
GET	/chambres?type=&batiment=&disponible=	Lister avec filtres optionnels	—
GET	/chambres/disponibles?type=&batiment=	Uniquement les chambres libres	—
GET	/chambres/:id	Détail d'une chambre	404
PATCH	/chambres/:id	Mise à jour partielle	404 · 409
DELETE	/chambres/:id	Suppression	404

3.3 Règles métier et détails

- **Une chambre = un résident** : deux méthodes dédiées `occuper()` / `liberer()` sont exposées au reste de l'application. `occuper()` réalise un `UPDATE … WHERE id = ? AND disponible = true` puis vérifie le nombre de lignes affectées : deux approbations concurrentes ne peuvent pas gagner. 404 si la chambre n'existe pas, 409 si elle est déjà prise.
- Validation des DTO par `class-validator` (pipe global en `whitelist + forbidNonWhitelisted + transform`).

- Documentation Swagger de tous les endpoints (`@ApiTags` , réponses typées).

4. Backend — Phase 2 : Demande Service

4.1 Entité `DemandeHebergement`

Champ	Type	Notes
`id`	uuid (PK)	
`etudiantId`	varchar(100), indexé	identifiant provenant (à terme) du JWT `IAuthentification`
`statut`	enum, indexé	`EN_ATTENTE | APPROUVEE | REFUSEE | ANNULEE`
`dateDepot`	timestamptz	défaut = maintenant
`dateArrivee / dateDepart`	timestamptz	`dateArrivee < dateDepart` obligatoire
`chambreId`	uuid nullable, indexé	chambre souhaitée puis attribuée

4.2 Cycle de vie (transitions autorisées)

```
EN_ATTENTE ──► APPROUVEE   (occupe la chambre, crée le contrat, émet demande.approuvee)
EN_ATTENTE ──► REFUSEE
EN_ATTENTE ──► ANNULEE
APPROUVEE  ──► ANNULEE     (libère la chambre, résilie le contrat, émet demande.annulee)
REFUSEE / ANNULEE : états terminaux
```

Toute autre transition, ou une transition vers le statut courant, renvoie **409 Conflict**. Une approbation exige une chambre libre (`chambreId` fourni dans le PATCH ou déjà souhaitée dans la demande) : la chambre est occupée de façon atomique avant l'enregistrement.

4.3 Endpoints

Méthode	Route	Description	Erreurs
POST	/demandes	Déposer une demande	400 dates invalides · 404 chambre souhaitée inconnue
GET	/demandes?statut=&etudiantId=&chambreId=	File admin (filtrable) ; l'étudiant est filtré sur son id par le front	—
GET	/demandes/:id	Détail	404
PATCH	/demandes/:id/statut	Changer le statut (approbation avec chambre, refus, annulation)	400 · 404 · 409

4.4 Événements émis

Événement	Charge utile	Consommateur

`demande.approuvee`	demandeId, etudiantId, chambreId, dateDebut, dateFin	ContratsService → création du contrat
`demande.annulee`	demandeId, chambreId	ContratsService → résiliation du contrat

5. Backend — Phase 3 : Contrat Service

5.1 Entité `Contrat`

Champ	Type	Notes
`id`	uuid (PK)	
`residentId`	varchar(100), indexé	
`chambreId`	uuid, indexé	
`demandeId`	uuid, indexé, **unique**	un seul contrat par demande
`dateDebut / dateFin`	timestamptz	`dateFin > dateDebut` ; `dateFin ≤ 31 juillet` de l'année académique
`montantMensuel`	numeric(10,2)	tarif par type : SIMPLE 150 · DOUBLE 100 · STUDIO 200 TND — transformer pour exposer un nombre
`statut`	enum, indexé	`ACTIF | TERMINE | RESILIE`

5.2 Règles métier

- **Création automatique** : le service écoute `demande.approuvee` et crée un contrat `ACTIF` en récupérant le type de la chambre pour le loyer. Opération **idempotente** (un contrat existe déjà pour la demande → on le retourne).
- **Dates** : `dateFin` strictement postérieure à `dateDebut` ; `dateFin` ne peut pas dépasser la fin de l'année académique (31 juillet, calculée depuis la date de début : rentrée à partir du 1er août → année suivante).
- **Résiliation** : seule une résiliation d'un contrat `ACTIF` est acceptée (sinon 409) ; la chambre associée est **libérée** via `ChambresService.liberer()`. L'annulation d'une demande approuvée résilie aussi le contrat (écoute de `demande.annulee`).
- **Robustesse** : les écouteurs d'événements capturent leurs erreurs et les journalisent (Logger) afin qu'un incident côté Contrat ne casse pas la réponse d'approbation de la Demande — cohérence à terme, typique d'une architecture événementielle (une réconciliation pourra être ajoutée).

5.3 Endpoints

Méthode	Route	Description	Erreurs
GET	/contrats?statut=&residentId=&chambreId=&demandeId=	Liste admin filtrable	—

GET	/contrats/resident/:residentId?statut=	Contrats d'un résident	—
GET	/contrats/:id	Détail	404
PATCH	/contrats/:id/resilier	Résilier (libère la chambre)	404 · 409

6. Backend — Structure, configuration et outillage

6.1 Arborescence complète (`backend/`)

```
backend/
├── docker-compose.yml          Postgres 16 + script d'init du schéma
├── scripts/
│   └── init-schema.sql         CREATE SCHEMA IF NOT EXISTS "BDD_Residences"
├── .env / .env.example         Port, préfixe API, connexion PostgreSQL, JWT
├── package.json                NestJS 10, TypeORM, class-validator, Swagger, event-emitter…
├── tsconfig.json / tsconfig.build.json / nest-cli.json
├── .eslintrc.js / .prettierrc / .gitignore
└── src/
    ├── main.ts                         Bootstrap : ValidationPipe global, Swagger /api/docs,
port 3001
    ├── app.module.ts                   ConfigModule + TypeOrmModule.forRootAsync + EventEmitter
+ 3 modules
    ├── config/
    │   ├── database.config.ts          Config typée (host, port, user, db, schéma…)
    │   └── ensure-schema.ts            CREATE SCHEMA IF NOT EXISTS avant la connexion TypeORM
    ├── common/
    │   ├── decorators/public.decorator.ts    @Public() (stub d'auth, branché plus tard)
    │   ├── guards/jwt-auth.guard.ts          Garde JWT « placeholder » (contrat
IAuthentification)
    │   └── transformers/numeric.transformer.ts  numeric PostgreSQL → number dans l'API
    ├── chambres/
    │   ├── chambres.module.ts / chambres.controller.ts / chambres.service.ts
    │   ├── chambres.service.spec.ts          13 tests unitaires (dont occuper/liberer)
    │   ├── entities/chambre.entity.ts · chambre-type.enum.ts
    │   └── dto/create-chambre.dto.ts · update-chambre.dto.ts · filter-chambre.dto.ts
    ├── demandes/
    │   ├── demandes.module.ts / demandes.controller.ts / demandes.service.ts
    │   ├── demandes.service.spec.ts          10 tests unitaires (transitions, événements)
    │   ├── entities/demande-hebergement.entity.ts · demande-statut.enum.ts
    │   ├── dto/create-demande.dto.ts · update-demande-statut.dto.ts · filter-demande.dto.ts
    │   └── events/demande.events.ts          Constantes + classes de charge utile
    └── contrats/
        ├── contrats.module.ts / contrats.controller.ts / contrats.service.ts
        ├── contrats.service.spec.ts          13 tests unitaires (règles, écouteurs, idempotence)
        ├── entities/contrat.entity.ts · contrat-statut.enum.ts
        ├── constants/pricing.constants.ts    Tarifs par type de chambre
        ├── utils/academic-year.util.ts       Fin d'année académique (31 juillet)
        └── dto/filter-contrat.dto.ts · find-resident-contrats.dto.ts
```

6.2 Bibliothèques utilisées

Bibliothèque	Rôle
@nestjs/{core,common,platform-express} 10	Framework HTTP, modules, pipes, guards
@nestjs/typeorm + typeorm + pg	ORM et pilote PostgreSQL (schéma `BDD_Residences`)
class-validator + class-transformer	Validation et transformation des DTO
@nestjs/swagger	Documentation OpenAPI interactive sur `/api/docs`

@nestjs/config	Chargement typé du `.env`
@nestjs/event-emitter	Bus d'événements interne (Demande → Contrat)
Jest + @nestjs/testing	Tests unitaires des 3 services (36 tests)

6.3 PostgreSQL

- Schéma physique dédié : `BDD_Residences`, créé automatiquement au démarrage (`src/config/ensure-schema.ts`) ou par `scripts/init-schema.sql` dans Docker.
- TypeORM en `synchronize: true` pour le développement (piloté par `DB_SYNCHRONIZE`) ; les migrations pourront remplacer ce mode plus tard.
- Un `docker-compose.yml` fournit Postgres 16 (base `smartcampus`, utilisateur `postgres`) — à utiliser sur une machine équipée de Docker.
- Variables : `DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, DB_SCHEMA, DB_SYNCHRONIZE, DB_LOGGING`.

6.4 Démarrage

```
cd backend
npm install
# 1) Démarrer PostgreSQL (docker compose up -d  OU  instance locale)
# 2) Configurer .env si besoin (défauts : localhost:5432 / postgres / smartcampus)
npm run start:dev        # API + Swagger sur http://localhost:3001/api/docs
npm run test             # suite Jest
npm run lint             # ESLint + Prettier
```

6.5 Vérifications effectuées

Résultats :

- `npm run build` (Nest) : compilation TypeScript sans erreur.
- `npm test` : **36 tests / 36 réussis** — Chambres 11, Demandes 12, Contrats 13.
- `npm run lint` : aucune erreur.
- `npm run build` (frontend) : `tsc -b` + build Vite réussis (chunk `HousingModule` généré).

Limite connue : aucun PostgreSQL n'étant disponible sur la machine (Docker absent, pas d'instance locale), les tests d'intégration HTTP réels contre la base n'ont pas pu être exécutés ; les tests unitaires mockent les repositories. La vérification de bout en bout se fera au premier `docker compose up -d` + `npm run start:dev`.

7. Frontend — Intégration au module Résidences

7.1 Ce qui existait / ce qui a changé

Élément	Avant (maquette)	Après (contrat backend)
Modèle	Residence (lits), HousingRoom (occupants), HousingRequest (4 étapes)	`Chambre` (type SIMPLE/DOUBLE/STUDIO, bâtiment, étage, disponible), `Demande` (statuts backend), `Contrat`
Routes pages	/residences, /residences/ma-demande, /residences/demandes, /residences/maintenance	/residences, /residences/chambres, /residences/demandes, /residences/demandes/new, /residences/contrats, /residences/maintenance
API	/residences, /housing-requests, /tickets (mocks seuls)	/chambres, /demandes, /contrats (appels réels NestJS) + mocks reproduisant les mêmes règles
Modèles supprimés	`ResidenceCard`, `RoomPicker`, `HousingRequestStepper`, `OccupancyChart`, pages `MyRequestPage` et `RequestsPage`	

7.2 Pages livrées

Route	Rôle(s)	Contenu
`/residences`	étudiant, admin	Tableau de bord : statistiques (chambres, libres, occupées, demandes en attente / mes demandes), occupation par bâtiment, dernières demandes, contrats actifs
`/residences/chambres`	étudiant, admin	Grille de chambres, filtres type / bâtiment / disponibilité ; admin : création (dialogue), bascule disponibilité, suppression
`/residences/demandes`	étudiant	Mes demandes : statut, séjour, chambre attribuée, annulation (EN_ATTENTE ou APPROUVEE)
`/residences/demandes`	admin	File complète : compteurs, filtre statut, tableau, approbation avec sélecteur de chambre libre (alerte 409 si occupée), refus
`/residences/demandes/new`	étudiant	Formulaire : date d'arrivée / de départ (validation arrivee < depart), chambre souhaitée optionnelle
`/residences/contrats`	étudiant	Détail de mes contrats : chambre, période, loyer mensuel, statut
`/residences/contrats`	admin	Tous les contrats, filtre statut, résiliation (confirmation, libère la chambre)
`/residences/maintenance`	étudiant, admin	Signalements de pannes (service mocké) : formulaire par bâtiment, traitement Signalé → En cours → Résolu

7.3 Correspondance front ↔ back

Fonction front (`features/housing/api`)	Route NestJS appelée

`listChambres` / `listChambresDisponibles`	GET /chambres · GET /chambres/disponibles
`createChambre` / `updateChambre` / `deleteChambre`	POST /chambres · PATCH /chambres/:id · DELETE /chambres/:id
`listDemandes` (étudiant filtré sur son id)	GET /demandes?statut=&etudiantId=
`createDemande`	POST /demandes
`approveDemande` / `refuseDemande` / `cancelDemande`	PATCH /demandes/:id/statut
`listContrats` / `listContratsResident` / `getContrat`	GET /contrats · GET /contrats/resident/:id · GET /contrats/:id
`resilierContrat`	PATCH /contrats/:id/resilier
`listTickets` / `createTicket` / `updateTicketStatus`	mock local uniquement (hors périmètre backend)

7.4 Mode démo (mocks) et mode réel

L'application partagée fonctionne par défaut avec `VITE_USE_MOCKS=true` (les autres modules de l'équipe ne disposent pas encore de backends). Le module Résidences respecte cette architecture : chaque fonction API fournit un `mock` exécutant les mêmes règles (occupation atomique simulée, transitions, création/résiliation de contrat, événements toastés via le bus de démo).

Pour brancher le backend NestJS :

```
# .env (racine du front)
VITE_USE_MOCKS=false
VITE_API_BASE_URL=http://localhost:3001/api
```

7.5 Fichiers ajoutés / modifiés / supprimés (frontend)

```
src/features/housing/
├── types/index.ts                [RÉÉCRIT]  Chambre, Demande, Contrat (+ Ticket), constantes,
tarifs
├── api/index.ts                  [RÉÉCRIT]  appels NestJS + mocks métier + événements de démo
├── mocks/index.ts                [RÉÉCRIT]  base en mémoire : 28 chambres (A/B/C), demandes,
contrats, tickets
├── hooks/index.ts                [RÉÉCRIT]  useChambres, useChambresDisponibles, useDemandes,
useContratsVisibles…
├── schemas/index.ts              [RÉÉCRIT]  zod : demande (+ dates), chambre (+ cohérence
capacité), ticket
├── i18n/index.ts                 [RÉÉCRIT]  libellés FR des nouvelles pages
├── README.md                     [MIS À JOUR] contrat API, règles, pages, branchement
├── utils/time.ts                 [MODIFIÉ]  + defaultDeparture, formatIsoDay
├── utils/students.ts             [AJOUTÉ]   noms d'affichage des matricules
├── components/
│   ├── DemandeStatusBadge.tsx    [AJOUTÉ]   badge En attente/Approuvée/Refusée/Annulée
│   ├── ContratStatusBadge.tsx    [AJOUTÉ]   badge Actif/Terminé/Résilié
│   ├── ChambreCard.tsx           [AJOUTÉ]   fiche chambre + actions admin
│   ├── ChambrePicker.tsx         [AJOUTÉ]   sélecteur de chambre libre (groupé par bâtiment)
│   ├── ChambreFormDialog.tsx     [AJOUTÉ]   dialogue de création de chambre (admin)
│   ├── BatimentOccupancy.tsx     [AJOUTÉ]   occupation par bâtiment (barres accessibles)
```

```
│   ├── TicketStatusBadge.tsx      [CONSERVÉ] badge de signalement
│   ├── ResidenceCard.tsx          [SUPPRIMÉ]
│   ├── RoomPicker.tsx             [SUPPRIMÉ]
│   ├── HousingRequestStepper.tsx  [SUPPRIMÉ]
│   └── OccupancyChart.tsx         [SUPPRIMÉ] (remplacé par BatimentOccupancy)
└── pages/
    ├── HousingModule.tsx          [RÉÉCRIT]  onglets + sous-routes du module
    ├── ResidencesPage.tsx         [RÉÉCRIT]  tableau de bord
    ├── ChambresPage.tsx           [AJOUTÉ]   liste + filtres + CRUD admin
    ├── DemandesPage.tsx           [AJOUTÉ]   vues étudiant (suivi/annulation) et admin
(approbation/refus)
    ├── NewDemandePage.tsx         [AJOUTÉ]   formulaire de demande
    ├── ContratsPage.tsx           [AJOUTÉ]   contrats étudiant / admin + résiliation
    ├── MaintenancePage.tsx        [ADAPTÉ]   résidence → bâtiment
    ├── MyRequestPage.tsx          [SUPPRIMÉ]
    └── RequestsPage.tsx           [SUPPRIMÉ]
```

Aucun autre module (Académique, Ressources, Bibliothèque, Vie étudiante, Utilisateurs, tableau de bord) n'a été touché : le registre `features/registry.ts`, la navigation et les routes existantes sont conservés tels quels — seul l'onglet du module Résidences pointe vers les nouvelles pages.

8. Limites et suites

- **Authentification** : le backend valide le format du Bearer token mais la vérification JWT contre le service Utilisateurs (`IAuthentification`) reste un stub (`JwtAuthGuard` + `@Public()`) — à activer quand le secret/JWKS partagé sera disponible. Les rôles sont pour l'instant appliqués côté front (registry) et simulés côté mocks.
- **Transactions** : approbation = occupation de la chambre puis création du contrat via événement. En cas d'échec du contrat, la demande reste approuvée (log d'erreur). Une saga/outbox ou une transaction partagée (même base) pourra renforcer ce point.
- **Intégration réelle PostgreSQL** : à exécuter dès que Docker ou une instance locale est disponible (création du schéma automatique au démarrage).
- **Maintenance** : hors périmètre des trois services, reste mockée avec un message explicite en mode réel (501).
- **Tests front** : l'application n'a pas de harnais de tests configuré ; la vérification s'appuie sur `tsc` + build Vite.

> **Résumé :** 3 microservices logiques NestJS livrés et testés (Chambre, Demande, Contrat), schéma PostgreSQL dédié, Swagger, bus d'événements interne, et les 5 pages demandées du module Résidences branchées sur ces API — le tout vérifié par compilation, 36 tests unitaires, lint et build frontend.